

Memory Management in Modern Operating Systems: Challenges and Innovations

Taylin Roberts

4/25/25

COSC 460

Abstract

This paper gives an analysis of memory management in modern operating systems, focusing on Windows. It goes in depth of concepts like virtual memory, paging, segmentation, and different memory allocation techniques. It highlights how these things enable secure and efficient multitasking. Comparing contiguous and non-contiguous allocation, and strategies like first-fit, best-fit, worst-fit, and the buddy system, evaluating their efficiency to prevent and impact on fragmentation. The paper also talks about challenges including fragmentation, memory leaks, security vulnerabilities, and how complex monitoring and troubleshooting can be. Finally, it considers recent technology trends-such as hybrid memory systems and real-time management tools-and discusses how Windows is adapting to ensure performance, reliability, and scalability.

Introduction

Memory management is a main function of most modern operating systems and is responsible for controlling and organizing the memory of a computer. The goal of this is for multiple programs to be able to run securely without interfering with each other. With more memory, a computer can handle more tasks at once. Things like getting on the internet, opening apps, or even watching a video need memory available to function properly.

Without proper memory management, crashes or performance bottlenecks can happen. The operating system uses the memory that is currently available most efficiently, so the computer runs smoothly and does not waste resources. Memory lets programs communicate with the hardware, tells programs to function, and helps tasks execute in the most efficient way.

This research paper will give a heavy overview of memory management in operating systems, with concepts like paging, virtual memory, memory allocation and segmentation. Next the paper will dive into challenges faced by operating systems designers such as fragmentation and memory leaks. The goal of this research paper is to compare and analyze different memory management strategies on operating systems such as Windows, to highlight the differences between each of their approaches. I will give an analysis of the modernity of how operating systems manage memory to show how new innovations are shaping the future of computing.

Core Concepts of Memory Management

Good memory management is the heart of all operating systems to ensure that many programs can run securely and smoothly. These systems rely on certain techniques and concepts to determine how memory is accessed and allocated. Some of these include virtual memory, paging and segmentation, where they each play a role in how applications will interact with hardware and how system performance will be optimized. This section on

the paper will be focused on exploring these concepts to understand some of the building blocks of memory management.

Virtual memory is a technique of memory management that lets the physical RAM use secondary storage such as Solid-State Drive, and Hard Disk Drives as an extension. Which then in turn makes an illusion of larger memory space for applications. This lets processes operate as if they have access to more memory resources when physical RAM is limited.

Virtual memory isolates each process's memory space which keeps applications from interfering with each other for seamless execution of computer programs. It is also efficient because data that is rarely used is moved to the disk temporarily to free the RAM for the active processes which prevents wasted memory for bigger workloads. When isolating memory, it prevents unauthorized access between processes which then reduces the risks of things like buffer overflow attacks.

Windows uses a hidden system file in the C: drive; pagefile.sys for its virtual memory.

Windows adjusts its size by default, but advanced users can do manual configuration. Too much use of this file can lead to slower disk I/O which can cause slowdowns compared to RAM. In Linux, implements swap spaces with files such as /swapfile. More modern kernels use zswap to better performance and zram (compressed RAM caching and in-memory compression), which reduces disk writes and betters responsiveness.

Paging splits physical and virtual memory into frames and pages. When memory is requested by a process, the operating system maps the logical pages to free frames with a page table. If a page is not in RAM, it gets loaded from the disk. Page tables store mappings

between physical and virtual addresses and the Memory Management Unit translates the addresses when it executes. If RAM is full, the operating system will evict pages using algorithms such as First in First Out; which evicts the oldest page, and Least Recently Used; which evicts the least recently used page.

Segmentation divides memory into segments based off of units such as code, stack, or data. These Segments match the structure of programs to reduce internal fragmentation. They also require dedicated registers to track the segment's base addresses and limits. Segmentation was a main part of earlier operating system designs but ended up being seen as complex to manage, while in modern systems segmentation is rarely used by itself because of fragmentation issues. Newer Combining segmentation and paging for a hybrid approach (segmented paging) to reduce memory fragmentation and make for a more efficient experience.

Memory Allocation Techniques

Efficient memory allocation is important to system performance and management of computer resource management. There are different techniques to allocate memory to processes and each has its pros and cons for different workloads.

With contiguous allocation, the processes are assigned a single, continuous block of memory. While this method is simple, it can lead to external fragmentation, where free memory is split into small pieces that can't be used. Examples include single contiguous

allocation and partitioned allocation, where memory is divided into fixed or variable-sized blocks. In non-contiguous allocation, a process's memory is spread across different locations in physical memory. Different techniques like paging and segmentation let processes use non-adjacent memory blocks, which in return helps to reduce external fragmentation and makes better use of all available memory.

The Buddy System is a dynamic memory allocation technique that divides memory into blocks of sizes that are powers of two. After a request is made, the system will find the smallest available block that fits and splits larger blocks into "buddies". When the memory is freed, the system puts the buddy blocks back together. This approach allows for fast allocation and deallocation and helps reduce fragmentation, but it can still cause internal fragmentation since allocations are rounded up to the nearest power of two.

Binary buddy heap before allocation:

64 kB free block

After allocating an 8 kB block:

8 kB allocated block	8 kB free block	16 kB free block	32 kB free block
----------------------------	--------------------	---------------------	---------------------

After allocating a 10kB block, with 6kB wasted because of rounding up:

8 kB allocated block	8 kB free block	10 kB allocated block	6 kB wasted block	32 kB free block
----------------------------	--------------------	-----------------------------	-------------------------	---------------------

First fit and best fit are strategies for allocating memory blocks to processes in allocation schemes.

First Fit allocates the first block of memory that is large enough to fulfill the request, it's faster but can cause fragmentation in memory. Best-Fit searches the memory to find the smallest block that fits the request, which reduces wasted space. However, it can be slower and may increase fragmentation over time. Worst-Fit allocates the largest available block, leaving sizable free blocks available for future allocations. This can also lead to inefficient memory use and fragmentation

Comparison of Efficiency and Fragmentation Handling:

Technique	Efficiency	External Fragmentation	Internal Fragmentation	Use Case
Contiguous	Fast/Simple	High	low	Simpler systems, embedded devices
Non- Contiguous	Moderate/Complex	Low	Sometimes (with paging)	Modern Operating Systems
Worst fit	Moderate	high	low	Rarely used
First Fit	Fast	Moderate	low	General purpose
Best Fit	Slow	High	low	Space constrained

Buddy System	Fast	Moderate	Moderate	dynamic
--------------	------	----------	----------	---------

Windows Memory Management

Windows memory management is built on an architecture that separates components of the system from user processes for performance and stability. Every Windows process has its own private virtual space that the operating system maps to physical memory with a linear memory model. The mapping gives processes access to memory while not needing to know the physical addresses. It also enables memory sharing and isolation between processes, and this virtual address space is split into user and kernel space.

Windows' virtual memory is managed through physical RAM and a pagefile.sys on disk. If the physical memory is exhausted, pages used less frequently get moved to the paging file

which lets the system continue running under the case of heavy memory loads. Windows 10 and later have memory compression which is where memory pages that are not used frequently get compressed in the RAM; and not paged out immediately to the disk. Memory compression betters the responsiveness by reducing the need of slower disk storage and using the RAM that is available better.

The working set is a core concept in Windows memory management; it is a set of memory pages actively being used by a process at a specific time. The operating system manages each process's working set to keep the most frequently used pages in RAM, while the least frequently used pages get moved out which optimizes performance and resource sharing. When trying access a page that's not in the memory currently a page fault will occur, and Windows responds to page faults by dispatching them to a memory manager fault handler. This gets the page from the other storage and then updates the working set. Using a combination of virtual memory management, page fault handling, and working set optimization, Windows gives responsive memory management for modern demands of computing.

Challenges in Modern Memory Management

Modern memory management in Windows has many technical and operational challenges that impact system stability, performance, and security. Fragmentation continues to be an

issue, with external fragmentation occurring when free memory is split into small, non-contiguous blocks, making it difficult to allocate large contiguous memory areas, and internal fragmentation happening when allocated memory blocks are not utilized fully, leading to wasted space within those blocks. Another challenge is memory leaks, in long-running processes and services. When applications fail to release memory that they no longer need it builds up over time, which reduces available memory for other processes and can force users to have to reboot their systems to restore performance.

Balancing system performance with resource limits is also an important concern. With Windows managing multiple applications and background services, it must pick which data to keep in fast physical memory and which data to move to slower virtual memory, especially as workloads grow complex and there are different hardware configurations. Security risks further complicate memory management. Weaknesses like buffer overflows and unauthorized memory access can be used by attackers, Microsoft reports that about 70% of security bugs in its products are related to memory safety issues. These bugs can lead to remote code execution, privilege escalation, and other severe consequences.

Space limitation is another challenge in modern memory management. 64-bit systems support much larger address spaces than 32-bit predecessors, but constraints on addressable memory and per-process address space still restricts the scalability and performance of applications that are memory-intensive. This is relevant for workloads that need large amounts of memory, such as large databases or scientific computing tasks. Another challenge is the efficient use of new storage technologies. With SSDs and other

high-speed storage devices becoming more widespread, Windows has to optimize the use of virtual memory and paging files to fully take advantage of these hardware advancements. Poorly managed paging can undermine performance but also speed up the wear on SSDs, shortening their lifespan.

Monitoring and troubleshooting memory issues become more complex and poses difficulties for users and administrators. As memory management gets better, diagnosing problems such as high memory usage, leaks, or abnormal patterns requires more advanced tools and a deeper comprehension of system internals. Utilities like Task Manager, Resource Monitor, and RAMMap give valuable insights, but interpreting their data and pinpointing the root cause of issues becomes challenging without specialized knowledge. These challenges highlight the need for more innovation in Windows memory management to maintain efficiency, and security in today's computing environments.

Innovations and Advancements

Windows has had significant growth in memory management and security. Memory compression introduced in Windows 10, lets the operating system store more data in the RAM by compressing less frequently used memory pages which in turn reduces how much data is written and read from the pagefile. This lets Windows have more applications in the memory at once and improves responsiveness.

Windows implements memory deduplication through memory combining and has been available since Windows 8.1 and Windows 10. This technique identifies identical memory pages across processes and merges them together into one physical page. Which results is a reduction in memory footprint, which is beneficial in environments with similar processes or virtual machines. Deduplication significantly optimizes memory usage and is important in server and virtualized settings.

Windows 11 strengthens memory security with hardware-backed protections. Core Isolation uses virtualization based security to isolate important system processes from the rest of the operating system, which reduces the risk of malware attacks. Memory Integrity is a part of Core Isolation and uses the Windows hypervisor to make sure that only trusted code runs in kernel mode, and that prevents unauthorized access and kernel-level exploits. These features provide defenses against memory-based attacks against the hardware.

Impact of Technology Trends

With the fast growth of AI, larger data applications is making the demand for memory on computing platforms very high. AI has a 70% increase in high bandwidth memory per year, and datacenter NAND usage is expected to grow up to 30% in the next year. Gaming platforms also push the need for better memory management as 16GB of RAM is the minimum for smooth gameplay now. Large-scale data applications and analytics need to have large and fast memory resources to process and store massive datasets in the most efficient way.

To keep up with all these things hybrid systems combining RAM and SSDs are becoming more common in today's world. This allows systems to use SSDs as an extension of RAM. Hybrid approaches, like those used in enterprise databases and cloud platforms, optimize performance by storing frequently accessed data in RAM while leaving SSDs for larger, less frequently used datasets. Windows is adjusting to these changes by building in things like memory compression, deduplication, and stronger hardware protections. It's also working with new memory tech like NVMe SSDs and persistent memory to boost performance for AI, gaming, and data-heavy programs. Plus, Windows management tools now give users and admins more ways to watch and control memory use in real time. All these updates help Windows stay ready for the future as apps keep getting bigger and more demanding.

Conclusion

Modern Windows memory management has evolved to address the complexity and demands of today's computing environments. Findings highlight how Windows uses a combination of physical RAM and virtual memory, dynamically allocating resources through mechanisms like working sets, paging, and smarter memory allocation strategies. Windows 11, has introduced improvements that better use SSDs for virtual memory, reduce system freezes, and make more intelligent decisions about which data should remain in fast physical memory or be moved to slower storage. Even with these advancements, there are still challenges. Internal and external fragmentation continue to be a problem, along with the risk of memory leaks in long-running processes. Balancing

performance with resource constraints can be very difficult as applications grow in complexity and size, with the rise of AI, gaming, and large-scale data processing. Security risks, such as buffer overflows and unauthorized memory access, continue to demand hardware-backed protections and improved isolation mechanisms. Additionally, the need to efficiently utilize new storage technologies and manage hybrid memory systems has its own ongoing technical challenges. Future directions for operating system memory managements are likely to focus on even tighter integration with emerging hardware, better support for hybrid and distributed memory architectures, and the application of AI-driven algorithms to further optimize memory allocation and usage. As datasets and application requirements continue to expand, the evolution of memory management will remain important to having fast, secure, and reliable computing experiences.

References

Citigroup. (n.d.). AI demand in the memory industry.

<https://www.citigroup.com/global/insights/ai-demand-in-the-memory-industry>

Coursera. (n.d.). Memory management. <https://www.coursera.org/articles/memory-management>

GeeksforGeeks. (n.d.). Memory management in operating system.

<https://www.geeksforgeeks.org/memory-management-in-operating-system/>

Learning Daily. (n.d.). Memory management in operating systems.

<https://learningdaily.dev/memory-management-in-operating-systems-4c9dd0017ce0>

MCCI. (n.d.). How to change the Windows pagefile size.

<https://mcci.com/support/guides/how-to-change-the-windows-pagefile-size/>

Microsoft. (n.d.-a). About memory management. Microsoft Docs.

<https://learn.microsoft.com/en-us/windows/win32/memory/about-memory-management>

Microsoft. (n.d.-b). Preventing memory leaks in Windows applications. Microsoft Docs.

<https://learn.microsoft.com/en-us/windows/win32/win7appqual/preventing-memory-leaks-in-windows-applications>

Microsoft. (n.d.-c). Troubleshoot issues with Performance Monitor. Microsoft Docs.

<https://learn.microsoft.com/en-us/troubleshoot/windows-server/support-tools/troubleshoot-issues-performance-monitor>

Stack Exchange. (2019, February 26). How to compute total internal and external fragmentation. <https://cs.stackexchange.com/questions/66011/how-to-compute-total-internal-and-external-fragmentation>

Stack Overflow. (2014, October 21). What is working set?

<https://stackoverflow.com/questions/896226/what-is-working-set>

TechInsights. (2023, March 14). AI continues to drive demand for memory solutions.

<https://www.techinsights.com/blog/ai-continues-drive-demand-memory-solutions>

TechTarget. (n.d.). What is memory management in a computer environment?

<https://www.techtarget.com/whatis/definition/memory-management>

The Memory Management Reference. (n.d.). Memory allocation.

<https://www.memorymanagement.org/mmref/alloc.html>

ZDNet. (2022, October 11). Microsoft: 70 percent of all security bugs are memory safety issues. <https://www.zdnet.com/article/microsoft-70-percent-of-all-security-bugs-are-memory-safety-issues/>

Report (PDF)

Assured Cloud Computing. (2015). VMDedup: Memory de-duplication in hypervisor [PDF].

University of Illinois. <https://assured-cloud-computing.illinois.edu/files/2015/08/VMDedup-Memory-De-duplication-in-Hypervisor.pdf>

GeeksforGeeks. (2024, March 8). Partition allocation methods in memory management.

<https://www.geeksforgeeks.org/partition-allocation-methods-in-memory-management/>